# 05_growth_b

## Retention

**Q26. Why do experienced growth people say retention is the most important part of the AARRR funnel, not acquisition?**

**Ideal answer.** Acquisition is a leaky-bucket problem: if retention is poor, every dollar of paid acquisition pours into a bucket that drains, so you scale spend to stand still and CAC payback never closes. Retention is the only stage that compounds - it simultaneously lifts LTV (you monetize a user over more periods), funds acquisition (a higher LTV/CAC ratio lets you bid more aggressively than competitors), and feeds Referral (retained users are the only ones around long enough to invite others). I treat retention as the multiplier on all four other AARRR stages rather than a co-equal stage, which is why the North Star for most subscription and marketplace businesses is a retained-usage metric (WAU, retained GMV), not a top-of-funnel count. The honest nuance is that for a low-frequency transactional business, "retention" reads as repeat-purchase rate and reactivation, not daily engagement, so the framing changes but the compounding logic does not. **Why Helios demonstrates it.** Helios models retention via `stats-mcp.cohort_retention` and `rfm_segment`, treating repeat-purchase/return-visit decay as a first-class diagnosable surface rather than a vanity acquisition count, consistent with the leaky-bucket view. **Follow-ups.** When would acquisition legitimately outrank retention as the priority? / How does a strong LTV/CAC ratio mechanically let you out-bid competitors on acquisition? / What North Star would you pick for a low-frequency e-commerce store specifically?

**Q27. Define a retention curve and explain what its shape tells you about product-market fit.**

**Ideal answer.** A retention curve plots the fraction of a cohort still active in period N against N (D1, D7, D30, or week/month buckets), anchored to a defined "active" event. The diagnostic signal is whether the curve **flattens** to a non-zero asymptote or **decays to zero**: a flat tail means you have a stable, habitual core of users who will never churn for organic reasons, which is the empirical fingerprint of product-market fit; a curve that asymptotes at zero means you have no durable base and are running pure acquisition arbitrage. I read three numbers: the D1/W1 drop (onboarding and first-value delivery), the slope of the middle (the habit-forming zone), and the height of the flattened tail (the size of your retained core). The classic mistake is comparing only headline retention rates between cohorts without aligning the "active" definition and the cohort-entry event, which makes two curves non-comparable. **Why Helios demonstrates it.** `stats-mcp.cohort_retention` computes period-N survival per cohort deterministically, so the curve shape (and any flattening) is a tool output, never an eyeballed estimate, satisfying the

determinism-where-it-matters principle. **Follow-ups.** How do you decide the "active" event for a transactional store vs a SaaS app? / What does a smiling (upward-sloping tail) retention curve indicate and how is it possible? / How would you statistically test whether two cohorts' tails differ?

**Q28. Walk me through the leaky-bucket framing and how you'd quantify the leak.**

**Ideal answer.** The leaky bucket models your active base as a tank: acquisition is the inflow, churn is the outflow through holes, and the water level is your active users. The level grows only when inflow exceeds outflow, so the first thing I quantify is the **net retention dynamic**: new + resurrected - churned per period. I size the leak as `churned_users / active_users_start_of_period` and immediately decompose it - new-user churn (a first-value/onboarding hole) behaves differently from established-user churn (a value-erosion hole), and they get different fixes. The framing forces the right prioritization question: a 5-point reduction in churn is usually cheaper and more compounding than the acquisition spend required to offset the same leak, because plugging the hole improves every future cohort, not just this one. **Why Helios demonstrates it.** Helios's funnel marts and `cohort_retention` let it attribute a level-change to inflow vs outflow per segment, and `decompose_change` separates whether a retention dip is a mix artifact (more low-retention traffic acquired) or a true rate leak (existing users churning faster). **Follow-ups.** How would you distinguish a leak caused by worse acquisition quality from one caused by product degradation? / What's the formula for the equilibrium active-user level given constant inflow and a constant churn rate? / Why might plugging the leak still not grow the base?

**Q29. How would you build a churn-prediction model, and what's the trap most people fall into?**

**Ideal answer.** I'd frame it as supervised classification: define churn precisely (e.g. no purchase in the next 90 days), build a feature set from RFM (recency, frequency, monetary), engagement trajectory, and tenure, then train a calibrated model (logistic regression or gradient-boosted trees) optimized for ranking quality - PR-AUC over accuracy, because churn is class-imbalanced. The pervasive trap is **target leakage**: including features that are mechanically downstream of churn (e.g. "days since last activity" measured into the prediction window) so the model "predicts" the outcome it's secretly observing - it scores beautifully offline and is useless in production. The second trap is optimizing AUC instead of **intervenable** churn: a model that flags users who were always going to leave has no business value; you want to surface the persuadable segment where an intervention changes the outcome, which is an uplift-modeling problem, not a pure-prediction one. **Why Helios demonstrates it.** Helios stays deliberately on the diagnostic side: `rfm_segment` produces interpretable, intervenable segments (e.g. "at-risk high-value: high monetary, decaying recency") that map directly to a prescribed action, rather than a black-box churn score with no lever attached - reflecting principle 4, every finding carries an action. **Follow-ups.** What's the

difference between a propensity-to-churn model and an uplift model? / How do you set the prediction-window length without leaking the label? / Why might a high-AUC churn model still lose money in production?

**Q30. What is resurrection (reactivation) and how do you measure whether a resurrection program is actually working?**

**Ideal answer.** Resurrection is bringing a churned/dormant user back to active status; in AARRR accounting it's a third inflow alongside new acquisition, and it's usually the cheapest growth dollar because the user already knows the product. I measure it as a cohort: take everyone dormant as of date T, apply the intervention, and track the **resurrection rate** = reactivated / dormant-targeted, but the number that matters is **second-life retention** - do resurrected users stick, or do they bounce back to dormant within a period? A program that resurrects 10% who all re-churn in 30 days is theatre. The correct evaluation is quasi-experimental: a holdout of dormant users who get no intervention, so you measure incremental resurrection (organic dormant users do reactivate on their own), and you net out that baseline before claiming credit. **Why Helios demonstrates it.** Helios's `rfm_segment` isolates the "lapsed" cells, and because the GA4 data is observational, Helios's `experiment-mcp` would **design and size** a resurrection holdout (power_analysis + runtime_estimate) and read it back as a pre/post difference-in-differences rather than asserting a raw before/after lift. **Follow-ups.** Why is a dormant-user holdout essential rather than a simple before/after? / How do you set the dormancy threshold that defines "resurrectable"? / What's the risk of over-messaging dormant users to resurrect them?

**Q31. Distinguish retention, churn, and engagement - candidates conflate these constantly.**

**Ideal answer.** Retention and churn are complements measured on a cohort over time - retention is the fraction still active in period N, churn is `1 - retention` for that period, and both require a fixed cohort and an "active" definition. Engagement is an **intensity** measure within the active population - sessions per user, items viewed, DAU/MAU stickiness ratio - and a user can be retained but disengaged (logs in but does little) or highly engaged right before churning (a usage spike can precede cancellation). The practical consequence is that engagement is a **leading indicator** of future retention but is not retention itself; I use engagement trends to forecast where the retention curve is heading and to find at-risk users before the churn event fires, but I never report rising engagement as if it were proof of improved retention. **Why Helios demonstrates it.** Helios tracks `engaged_sessions` and `engagement_rate` as distinct metrics from session/return-visit retention in the semantic registry, keeping intensity and survival separate so the Diagnose agent can flag "engagement falling ahead of retention" as a leading-indicator finding. **Follow-ups.** Give an example where engagement rose but retention fell - what's the explanation? / Why is DAU/MAU a stickiness measure and not a retention measure? / How would you use an engagement leading indicator to trigger an intervention?

**Q32. How do you choose retention period buckets (D1/D7/D30 vs W1/W4/M3) for a given product?**

**Ideal answer.** The bucketing must match the product's **natural usage frequency** - the cadence at which a satisfied user would naturally return. A daily-habit product (social, news) uses D1/D7/D30 because a day is a meaningful unit; a weekly product (grocery, fitness) uses week buckets; a low-frequency transactional store like the GA4 Merch Store uses month or even quarter buckets because expecting a D1 return is nonsensical and would manufacture an artificially brutal curve. The test I apply: pick the bucket where the median active user's inter-event gap fits comfortably inside one bucket, so a single skipped period isn't immediately misread as churn. Getting this wrong is a silent error - too-tight buckets make a healthy low-frequency product look like it's hemorrhaging, and too-loose buckets hide a real daily-product problem until it's catastrophic. **Why Helios demonstrates it.** On the GA4 Merch Store's transactional, low-frequency pattern Helios uses return-visit / repeat-purchase windows appropriate to that cadence rather than DAU-style daily retention, and is explicit (a maturity point) that the ~3-month dataset window caps how long a bucket horizon it can honestly report. **Follow-ups.** What inter-event-gap distribution would justify switching from weekly to monthly buckets? / How does the 3-month GA4 window constrain your bucket choice? / How do you handle a product with two distinct usage frequencies (power users vs casual)?

**Q33. A cohort's retention looks flat overall but is collapsing in one segment. How does Simpson's paradox apply, and how do you catch it?**

**Ideal answer.** Aggregate retention `R = Σ w_i · r_i` is a volume-weighted blend of per-segment retention rates, so a severe collapse in one segment can be masked at the top line by a simultaneous **mix shift** toward a higher-retaining segment - the classic Simpson's-paradox structure. To catch it I never trust the aggregate alone; I decompose the change into a **mix effect** ( `Σ Δw_i · r_i` , composition moved) and a **rate effect** ( `Σ w_i · Δr_i` , in-segment behavior moved), and I drill the rate effects because those are the real behavior change. A flat aggregate with a large negative rate effect in one segment offset by a positive mix effect is exactly the failure that a dashboard's single retention number hides, and it's the situation that costs analysts a day of manual slicing. **Why Helios demonstrates it.** This is the literal centerpiece: `stats-mcp.decompose_change` splits `ΔR` into mix/rate/interaction so a hidden segment collapse surfaces as a rate effect even when the top-line is flat - the exact Simpson's-paradox dissolution Helios was built around. **Follow-ups.** Write the three decomposition terms and state which one you'd drill first. / How do you guard against computing the aggregate rate as a mean of per-segment ratios? / What does a large interaction term tell you mechanically?

**Q34. How would you set a retention target and tie it to a North Star metric without gaming it?**

**Ideal answer.** I anchor retention to a North Star that captures **delivered value**, not activity - retained-usage or retained-revenue, something a user would only do if they got real value (e.g. "monthly active purchasers" rather than "monthly logins"). The target is set off the retention curve's flattening point: I commit to lifting the **asymptotic tail** (the durable core) by N points, not just the easily-gamed D1 number, because early-period retention can be juiced with notification spam that erodes the tail. To prevent gaming I pair the headline target with a guardrail metric - e.g. unsubscribe/complaint rate or per-user value - so a "win" that's actually annoyance-driven is caught. Frameworks: North Star + input metrics tree, and I prioritize the candidate retention levers with ICE/RICE so effort goes to high-confidence, high-reach interventions first. **Why Helios demonstrates it.** Helios's Prescribe agent emits **ICE/RICE-scored, powered** experiment cards for retention levers and the Critic refutes findings that would game a metric (insufficient sample, seasonality, data-quality), so a target is only "moved" by a defensible, guardrail-aware intervention. **Follow-ups.** What guardrail metric pairs with a D1-retention target? / Why is the tail height a better target than the D1 drop? / How would RICE rank a notification change vs an onboarding redesign?

## Referral

**Q35. Define the K-factor (viral coefficient) and explain what value range actually matters in practice.**

**Ideal answer.** K-factor = `i · c`, where `i` is the average number of invites each user sends and `c` is the conversion rate of an invite into a new active user; it's the expected number of new users each existing user begets through referral. The thresholds: K >= 1 means self-sustaining exponential growth (each cohort more than replaces itself virally), which is extraordinarily rare and usually short-lived; the realistic and still-valuable regime is **K < 1 but meaningful** (say 0.4-0.7), where virality acts as an **acquisition multiplier** on every other channel - a paid user who refers 0.5 others cuts your effective blended CAC by a third. The number people forget is the **viral cycle time**: a K of 0.5 with a one-day loop massively outperforms K of 0.7 with a 30-day loop, because growth compounds on cycles, not on K alone. **Why Helios demonstrates it.** Helios frames referral as a measurable loop with `i` and `c` as governed metrics in the funnel, so a referral diagnosis decomposes a K-factor move into "fewer invites sent" vs "lower invite conversion" - the same numerator/denominator discipline as the rest of the funnel. **Follow-ups.** Why is a sustained K >= 1 almost never observed in real products? / How does viral cycle time change the prioritization between lifting i vs c? / How would you measure K when invites and signups are attributed imperfectly?

**Q36. Walk through the standard growth loops and how a referral loop differs from a viral loop.**

**Ideal answer.** A growth loop is a closed system where the output of one cycle becomes the input to the next, so growth reinvests in itself rather than relying on linear top-of-funnel spend; the canonical types are viral/referral loops, content/SEO loops, and paid loops (revenue funds acquisition that generates more revenue). The distinction people blur: a **viral loop** is intrinsic - the product is more valuable when shared, so usage itself generates invites (a shared doc, a multiplayer game), and the invite is the product; a **referral loop** is extrinsic - you bolt an incentive ("give $10, get $10") onto a product that works fine solo, and the loop runs on the incentive economics, not the product mechanic. The consequence is that viral loops are cheap and durable but only available to inherently-social products, whereas referral loops are bolt-on-able to almost anything but cost margin and degrade when the incentive stops. **Why Helios demonstrates it.** Helios itself runs on a loop framing - the action-tracking memory closes a "did-the-fix-work" loop where each diagnosis's prescribed action feeds back as a measured input to the next run, mirroring how a growth loop reinvests output as input. **Follow-ups.** Which loop type would you prioritize for a single-player productivity tool? / How do you detect when a referral loop's incentive is being arbitraged/fraudulently exploited? / Why do paid loops eventually saturate while content loops can compound longer?

**Q37. How do you connect NPS to actual referral behavior - and why is NPS alone insufficient?**

**Ideal answer.** NPS measures stated willingness to recommend (promoters minus detractors on a 0-10 scale), but stated intent is not realized behavior, so I treat NPS as a **leading proxy** and validate it against the actual referral funnel: do self-identified promoters in fact send more invites with higher conversion? The right pipeline is to instrument the **behavioral** referral funnel - invites sent, invites accepted, referred-user activation - and correlate it back to survey segments, because the gap between "would recommend" and "did recommend" is where the real lever lives (often a missing or high-friction sharing mechanic, not a satisfaction problem). NPS alone is insufficient because it's sample-biased (who responds), it's a lagging point-in-time read, and a high NPS with no sharing affordance produces zero actual referrals; conversely you can engineer referrals from passives with good incentives. I use NPS to size the *latent* promoter pool and the behavioral funnel to find why that pool isn't converting to invites. **Why Helios demonstrates it.** Helios privileges behavioral funnel evidence over stated metrics throughout - every finding must be SQL-verified against the governed event funnel and carry a significance test, so a "promoters drive referral" claim would have to be backed by the invite-conversion data, not the survey alone. **Follow-ups.** How would you statistically test that promoters refer more than passives? / What product change closes a high-NPS, low-referral gap? / Why is NPS response bias dangerous when sizing the promoter pool?

**Q38. A referral program's signups jumped but revenue didn't. How do you diagnose it?**

**Ideal answer.** This is a classic quality-vs-quantity referral failure, and I'd decompose revenue-per-referred-user into its parts: `revenue = referred_signups · activation_rate · purchase_rate · AOV`. A signup spike with flat revenue means the gain is concentrated upstream and lost downstream - typically the incentive attracted **incentive-seekers** (low purchase intent) or fraud, so activation and purchase rates collapsed in the referred cohort even as the count rose. I'd cohort referred vs organic users on downstream conversion and check whether the new referred mix is dominated by a low-value segment - i.e. is this a **mix shift** toward low-quality traffic (a composition artifact) rather than a real behavior change? The fix follows the diagnosis: if it's incentive-seeker mix, restructure the incentive to reward a *purchase* not a *signup*; if it's fraud, add verification gates. **Why Helios demonstrates it.** Helios's `decompose_change` would split the referred-cohort revenue move into mix (more low-value referred traffic) vs rate (referred users converting worse) - so it distinguishes "we acquired junk" from "the product broke for referred users," and prices the gap as revenue-at-risk in dollars. **Follow-ups.** What incentive structure reduces incentive-seeker abuse? / How do you separate referral fraud from genuinely low-intent referred users? / If it's a mix shift, why might the program still be worth keeping?

### Q39. What metrics define a healthy referral funnel, and where do referral programs most commonly leak?

**Ideal answer.** The referral funnel is its own AARRR mini-funnel: share-prompt impressions -> invites sent ( `i` ) -> invites delivered -> invites accepted/converted ( `c` ) -> referred-user activation -> referred-user retention. I'd track step-to-step rates and the end-to-end **referred-user value** so the program is judged on retained revenue, not raw signups. The most common leaks are early: the share prompt is buried or mistimed (low invites-sent), or the invite landing experience is generic and high-friction (low conversion). The subtler leak is at the tail - referred users activate but don't retain because the incentive, not the value proposition, drove them in, so they never reach the habit zone. Healthy looks like a non-trivial invites-per-user, double-digit invite conversion, and referred-user retention at parity with or above organic. **Why Helios demonstrates it.** Helios treats every funnel as step-to-step governed rates with `reached_*` monotonic flags, so a referral funnel slots into the same machinery - the Diagnose agent drills the lowest-yielding step and the Critic checks the referred-cohort sample is large enough to trust. **Follow-ups.** Which referral-funnel step would you instrument first if you could only add one event? / Why judge a referral program on referred-user retention rather than signups? / What does referred-user retention below organic tell you?

### Q40. How do you attribute a referred user's value back to the referrer, and why is double-counting a risk?

**Ideal answer.** Attribution needs a referral edge that links referrer -> referred at signup (a referral code/link), then I credit the referred user's downstream value to the referrer's account for program ROI, while being careful that the **referred user's own value is counted once** in

company-level revenue - the referrer credit is a *program-accounting* overlay, not additional revenue. Double-counting bites two ways: counting the referred user's revenue both as "organic" and "referral-driven" inflates total acquisition impact, and in multi-touch journeys a user touched by paid *and* referral can be claimed fully by both channels, summing attributed revenue above 100%. I enforce a single attribution model (e.g. first-touch or a fractional model) consistently across channels so the channel-level numbers reconcile to the company total. **Why Helios demonstrates it.** Helios's reconciliation gate ( `warehouse-mcp.reconcile` , drift > 0.5% fails the finding) is exactly the defense against attribution double-counting - any channel-level revenue breakdown must sum back to the canonical total, so an over-100% attribution would fail before it ships. **Follow-ups.** First-touch vs fractional attribution for referral - when does each mislead? / How does the GA4 first-touch traffic_source gotcha complicate referral attribution? / How would you reconcile channel-attributed revenue to the company total?

**Q41. When is a referral program the wrong growth lever to invest in?**

**Ideal answer.** Referral is the wrong lever when the **prerequisite isn't met**: you need a retained, satisfied core first, because referral amplifies whatever you have - a great product spreads, a mediocre one spreads its dissatisfaction, and incentivizing referrals of a leaky product just accelerates churn-on-arrival. It's also wrong for low-frequency, low-emotional-salience purchases where users have few natural moments to share and little reason to (most people don't evangelize a one-off commodity purchase), and for products where the unit economics can't absorb a two-sided incentive without going underwater on referred-user LTV. The framework check: confirm retention/PMF first (flattening retention curve), confirm a natural sharing moment exists, and confirm referred-user LTV exceeds the incentive cost - fail any and the spend belongs in retention or a different acquisition channel. **Why Helios demonstrates it.** Helios's maturity stance applies here - it would refuse to prescribe a referral push as a top experiment if the diagnosis shows the leak is in-funnel rate decay (a retention/conversion problem), routing effort to the highest-impact lever via ICE/RICE rather than defaulting to a fashionable tactic. **Follow-ups.** What retention threshold would you require before launching referral? / Why can referral accelerate the death of a poor product? / How do you sanity-check that referred-user LTV exceeds the two-sided incentive cost?

**Q42. How would you design an experiment to measure the true incremental impact of a referral incentive?**

**Ideal answer.** I'd run a randomized holdout: split eligible users into a treatment group that sees the incentive and a control that doesn't, then measure incremental referred-user acquisition and, crucially, incremental **retained revenue** net of incentive cost - because some referrals would have happened organically and naive before/after over-credits the program. I'd size it with a power analysis on the expected lift and baseline variance to get a minimum detectable effect and a

runtime, and I'd pre-register the success metric and guardrails (referred-user retention parity, fraud rate) to avoid post-hoc cherry-picking. Where a clean randomized split is impossible, I fall back to a quasi-experiment - difference-in-differences against a comparable un-incentivized segment - and state the parallel-trends assumption explicitly. **Why Helios demonstrates it.** This is precisely `experiment-mcp`'s job: `power_analysis` + `runtime_estimate` + `design_experiment` produce a powered card, and because the GA4 data is observational, Helios is honest that it **designs and sizes** the test and reads it back quasi-experimentally (pre/post, diff-in-diff) rather than claiming to have run a live A/B. **Follow-ups.** Why does a before/after comparison over-credit the incentive? / How do you pick the minimum detectable effect for the power analysis? / What's the parallel-trends assumption and how would you check it?

## Revenue

### Q43. Walk me through revenue-per-session as a decomposition - why is it more useful than tracking revenue directly?

**Ideal answer.** Revenue-per-session decomposes cleanly as `RPS = session_conversion_rate · AOV` - the probability a session buys, times the average dollars when it does. This is far more useful than a raw revenue line because revenue conflates volume, conversion, and basket size, so a revenue dip is ambiguous; RPS isolates **yield per unit of traffic**, and the two-factor split tells you immediately whether a movement is a **conversion** problem (fewer sessions buying) or a **monetization** problem (buyers spending less), which have completely different fixes. I extend it with `AOV = items_per_transaction · price_per_item` when I need to know whether basket size or unit price moved. The discipline is to always compute the rate as `SUM(revenue)/SUM(sessions)` after grouping, never an average of per-segment RPS, or Simpson's paradox creeps back in. **Why Helios demonstrates it.** `revenue_per_session` is a first-class governed metric in Helios's semantic registry, and the RPS = conversion x AOV identity is exactly the kind of multiplicative decomposition the Decompose agent runs so a revenue move is attributed to conversion vs basket before any narrative is written.

```sql
-- governed shape: RPS = conversion_rate * AOV, computed as ratios of sums
SELECT
  channel_group,
  SAFE_DIVIDE(SUM(revenue), SUM(sessions))                AS revenue_per_session,
  SAFE_DIVIDE(SUM(purchasing_sessions), SUM(sessions))    AS session_conversion_rate,
  SAFE_DIVIDE(SUM(revenue), SUM(transactions))            AS aov
FROM helios.marts.fct_daily_funnel
GROUP BY channel_group
```

**Follow-ups.** If RPS fell, how do you tell a conversion problem from a basket problem? / Why compute the rate as a ratio of sums rather than averaging segment RPS? / How would you decompose AOV one level further?

**Q44. Define ARPU, AOV, and LTV and explain how they relate - candidates mix these up constantly.**

**Ideal answer.** AOV (average order value) = `revenue / transactions` - a per-**order** measure, blind to how often a user orders. ARPU (average revenue per user) = `revenue / users` over a period - a per-**user** measure that folds in purchase frequency, so `ARPU ~= AOV · orders_per_user`. LTV (lifetime value) is the **cumulative discounted** revenue (or margin) a user generates over their entire relationship - effectively ARPU integrated over the retention curve, `LTV ~= ARPU_per_period · expected_lifetime`, where expected lifetime is driven by retention/churn. The relationship chain is the key insight: AOV is a single-transaction lens, ARPU adds frequency, and LTV adds longevity - so to grow LTV you can lift basket (AOV), buy more often (frequency), or churn less (retention), and retention is usually the highest-leverage of the three because it multiplies the whole stream. **Why Helios demonstrates it.** Helios carries `aov` and `revenue_per_user` (ARPU) as distinct governed metrics and is explicit (a maturity point) that true LTV is **deferred** - the ~3-month GA4 window can't observe a real lifetime - so it reports short-horizon ARPU-based proxies with stated assumptions rather than fabricating an LTV number. **Follow-ups.** Why is AOV alone a misleading health metric? / How does the discount rate enter the LTV formula and why does it matter? / Which of basket, frequency, or retention would you prioritize to grow LTV and why?

**Q45. Why is Helios honest that LTV is "deferred," and how would you compute a defensible LTV proxy on a 3-month window?**

**Ideal answer.** LTV requires observing a full customer lifetime, but the GA4 Merch Store dataset spans only ~Nov 2020-Jan 2021, so any "LTV" computed on it is really a 90-day cumulative revenue truncated long before the retention curve flattens - reporting it as LTV would massively understate value and bake in a hidden truncation bias. The defensible move is to report a clearly-labeled **short-horizon proxy** - e.g. cumulative revenue-per-acquired-cohort at 30/60/90 days - and state the assumption explicitly, or fit a model (BG/NBD + Gamma-Gamma for transactional, or a survival-curve extrapolation) and surface the **confidence interval and the extrapolation assumption** rather than a false point estimate. The senior signal is naming the limitation: a fabricated LTV is worse than an honest 90-day proxy because it drives real CAC-payback decisions off a number that can't be supported. **Why Helios demonstrates it.** Refusing to fabricate LTV from a 3-month window is one of Helios's explicit honesty/maturity points - it reports short-horizon proxies with stated assumptions, which reads as senior judgment, not a gap. **Follow-ups.** What bias does truncating the retention curve introduce into a naive 90-day "LTV"? / When would a BG/NBD model be appropriate vs a simple cohort cumulative? / How would you communicate the proxy's uncertainty to a non-technical exec?

**Q46. What is expansion revenue and net revenue retention, and why do investors weight NRR so heavily?**

**Ideal answer.** Expansion revenue is additional revenue from **existing** customers - upsell, cross-sell, usage growth, larger baskets over time - as opposed to new-logo revenue. Net revenue retention (NRR) = `(starting_revenue + expansion − contraction − churn) / starting_revenue` for a cohort; NRR > 100% means your existing base grows revenue even with zero new acquisition, which is the holy grail because it means the leaky bucket actually *fills itself*. Investors weight NRR heavily because it's the cleanest single read on durable, compounding, capital-efficient growth: a company at 120% NRR has a built-in growth engine and predictable revenue, whereas one relying entirely on new logos to offset churn is on a treadmill. For e-commerce specifically, expansion reads as rising repeat-purchase frequency and basket growth within a retained cohort rather than seat/usage upsell. **Why Helios demonstrates it.** Helios cohorts revenue with `cohort_retention` and `rfm_segment`, so it can in principle measure within-cohort revenue growth (expansion via rising frequency/AOV) vs contraction - the building blocks of an NRR read - though it's honest the short window limits the horizon over which expansion can be observed. **Follow-ups.** Why can NRR exceed 100% even while logo churn is positive? / What does expansion revenue look like for a transactional e-commerce store vs SaaS? / How does the 3-month window constrain measuring expansion?

**Q47. How would you approach a pricing change analysis, and what's the elasticity trap?**

**Ideal answer.** Pricing is the highest-leverage revenue lever because it flows almost entirely to margin, but it's the most dangerous because it moves demand. I'd estimate **price elasticity of demand** ( `%Δ quantity / %Δ price` ) per segment, because elasticity is rarely uniform - price-sensitive segments shrink while inelastic ones don't - and revenue impact = the net of the price lift times the volume contraction. The elasticity trap is treating it as a single global constant and extrapolating linearly: elasticity is local (valid near the current price, not for a 2x move), it's confounded by promotions/seasonality/competitor moves, and a naive before/after read attributes seasonal demand swings to the price change. The correct approach is a controlled price test (randomized by cohort or geo) or, failing that, a quasi-experimental diff-in-diff with an explicit control, and I'd report the revenue and margin impact with the volume-contraction risk, not just the per-unit gain. **Why Helios demonstrates it.** Helios separates seasonality from real change via the Critic's seasonality-decoy refutation and Decompose's mix/rate split, so a "price change worked" claim is attacked for whether the revenue move is just a seasonal swing - directly defusing the elasticity-confounding trap. **Follow-ups.** Why is global constant elasticity a dangerous assumption? / How would you isolate the price effect from a concurrent seasonal demand swing? / What guardrail tells you a price rise is destroying long-run value despite short-run revenue gains?

**Q48. Revenue dropped 12% week-over-week. Give me the structured framework you'd use to diagnose it.**

**Ideal answer.** I'd decompose top-down, never guess. First the **factor split**: revenue = sessions x conversion_rate x AOV, so I attribute the 12% to traffic volume, conversion, or basket - that alone usually halves the search space. Then within whichever factor moved, a **segment decomposition** via mix/rate: is conversion down because traffic shifted toward a low-converting channel/device (mix effect, a composition artifact) or because a specific segment's conversion genuinely fell (rate effect, real behavior)? I drill the **rate effects first** because those are the actionable behavior changes, and I check seasonality and data-quality (a tracking break or a duplicated/lost shard can fake a 12% drop) before declaring a behavioral cause. Finally I **price it** in dollars of revenue-at-risk and attach a significance test so I'm not chasing noise, then prescribe the highest-ICE fix. **Why Helios demonstrates it.** This is literally the Helios pipeline end-to-end: Monitor detects the anomaly, Decompose runs the factor and mix/rate split, Diagnose does best-first RCA, Critic refutes seasonality/data-quality/sample confounds, and Narrator ships a dollar-quantified Decision Brief - the structured framework executed autonomously. **Follow-ups.** Which factor (traffic/conversion/AOV) would you check first and why? / How do you rule out a tracking/data-quality artifact before blaming behavior? / How does pricing the drop in revenue-at-risk change the prioritization?

**Q49. What is revenue-at-risk and why insist every finding carries a dollar figure?**

**Ideal answer.** Revenue-at-risk is the dollar magnitude of a diagnosed movement - the counterfactual revenue you'd have had absent the anomaly minus the actual, summed over the affected window (and projected forward if the cause persists). I insist on it because a finding without a dollar figure can't be **prioritized**: "checkout conversion dropped in Paid Search mobile" is interesting, but "that's costing $14k/week and growing" is what decides whether it jumps the backlog. Dollarizing also forces honesty - it converts a statistically-significant-but-tiny effect into a visibly trivial number, killing the trap of chasing significant-but-immaterial findings, and it makes the prioritization framework (ICE/RICE) operate on real impact rather than vibes. The figure must be derived deterministically from reconciled revenue, not estimated in prose, or it's just a confident-sounding guess. **Why Helios demonstrates it.** Principle 4 - "every finding carries a significance test, a dollar impact, and an action" - is non-negotiable in Helios; revenue-at-risk is computed from governed, reconciled revenue (drift > 0.5% fails) and is the field that lets Prescribe rank the experiment backlog by real money. **Follow-ups.** How do you compute the counterfactual baseline for revenue-at-risk? / Why does dollarizing protect against chasing statistically-significant-but-trivial effects? / Should revenue-at-risk be the realized loss or the projected-forward loss, and when?

**Q50. How do conversion-rate optimization and AOV optimization trade off, and how do you decide which to pursue?**

**Ideal answer.** Both lift RPS (since `RPS = conversion x AOV`) but through opposite mechanics that can fight each other: conversion optimization removes friction and lowers the bar to buy (simpler checkout, lower entry price, free shipping) which can *drag AOV down* by pulling in smaller baskets; AOV optimization (upsells, bundles, minimum-for-free-shipping thresholds) adds steps or raises the bar and can *suppress conversion*. So I never optimize one in isolation - the objective function is **revenue-per-session**, and I evaluate any change on RPS net, watching the second factor as a guardrail. To decide which to pursue, I look at the decomposition and elasticity: if conversion is far below benchmark and baskets are healthy, the marginal dollar is in conversion; if conversion is near-ceiling and baskets are thin, it's in AOV - and I size each candidate's RPS impact and rank with ICE/RICE rather than following a default playbook. **Why Helios demonstrates it.** Because Helios decomposes `revenue_per_session` into conversion x AOV, a finding can say *which* factor is the binding constraint and Prescribe can target the lever with the larger RPS upside - and the Critic guards against a "conversion win" that's actually cannibalizing AOV by demanding the net RPS effect, not a single-factor lift. **Follow-ups.** Give a concrete change that lifts conversion but lowers AOV - is it net positive? / Why is revenue-per-session the right objective function rather than either factor alone? / How would you size the RPS impact of a free-shipping-threshold change before shipping it?